JENKINS COMPLETE NOTES

PART 3 — JOBS & SOURCE CODE MANAGEMENT

Freestyle Jobs • Git • GitHub Integration • Branches • Webhooks • Build Triggers

Beginner-friendly explanations, practical project examples, troubleshooting steps, and scenario-based interview questions with answers.

Designed for Jenkins and DevOps interview preparation

How to use these notes

Each topic follows the same five headings: Definition, Purpose, Working, Practical Example, and Scenario-Based Interview Questions and Answers. The examples use a Windows Jenkins environment and an EKS Terraform repository where relevant.

1. Freestyle Jobs in Jenkins

1. Definition — What is it?

A Freestyle Job is a basic Jenkins job used to automate tasks such as downloading source code, executing shell or batch commands, building applications, running tests, and deploying applications. It is configured mainly through the Jenkins web interface.

2. Purpose — Why do we use it?

- Automate repetitive manual tasks.
- Build and test applications.
- Execute Linux shell scripts or Windows batch commands.
- Integrate Jenkins with GitHub.
- Perform basic deployment and maintenance tasks.

3. Working — How does it work?

Create a job from Jenkins Dashboard → New Item.

Configure Source Code Management to connect a GitHub repository and select a branch.

Add build steps, such as shell commands, Windows batch commands, or build tools.

Run the job and review its console output and result.

4. Practical Example — How do we use it in a project?

Open Jenkins at `http://localhost:8180`, click **New Item**, enter `Freestyle-GitHub-Test`, and select **Freestyle project**.

Under Source Code Management, select Git, enter the repository URL, and choose the required branch, such as `*/master`.

Under Build Steps, choose **Execute Windows batch command** if Jenkins runs on Windows.

```
echo Starting Jenkins build
git --version
terraform --version
echo Build completed successfully
```

Click **Save** → **Build Now**, then open **Console Output**. Git and Terraform must be available to the Windows account running the Jenkins agent.

5. Scenario-Based Interview Questions and Answers

Q1. You need to run a simple script every day at midnight. Which Jenkins job would you choose?

Answer: Use a Freestyle Job and configure Build periodically with `H 0 * * *`. This schedules a run approximately once per day around midnight. Jenkins uses H to distribute execution times.

Q2. Your Freestyle Job fails with 'terraform is not recognized'. What will you do?

Answer: Check Terraform installation, run `terraform --version` in the Jenkins execution environment, verify PATH for the Jenkins service account, restart the Jenkins service if needed, and rerun the job.

Q3. The build succeeds, but the application is not deployed. Why?

Answer: A successful build does not prove deployment occurred. Check that a deployment step is configured, review its output, and verify deployment credentials, target environment, and application health.

2. Git — Source Code Management

1. Definition — What is it?

Git is a distributed version control system used to track changes in source code and other project files. It maintains history and supports branches, comparisons, collaboration, and restoration of earlier versions.

2. Purpose — Why do we use it?

- Track code changes and history.
- Restore previous versions of files.
- Collaborate with multiple developers.
- Create branches for features and bug fixes.
- Integrate source code with Jenkins CI/CD pipelines.

3. Working — How does it work?

Modify files in the working directory.

Use `git add` to stage selected changes.

Use `git commit` to save a snapshot in the local repository.

Use `git push` to upload commits to a remote repository such as GitHub.

4. Practical Example — How do we use it in a project?

Suppose you update a Terraform configuration in your local repository. Run these commands:

```
git status
git add eks-cluster.tf
git commit -m "Update EKS cluster configuration"
git push origin master
```

`git status` checks modified files; `git add` stages changes; `git commit` saves local changes; `git push` uploads commits.

To download and integrate remote changes, use `git pull origin master`. To inspect branches, use `git branch` and `git branch -a`.

5. Scenario-Based Interview Questions and Answers

Q1. You committed changes, but Jenkins is building old code. What will you check?

Answer: Confirm the commit was pushed to the correct remote branch, verify the branch configured in Jenkins, inspect the commit hash in the console log, and ensure the repository URL is correct.

Q2. You modified the wrong file. How can you identify changes before committing?

Answer: Run `git status` to list modified and staged files and `git diff` to view unstaged changes.

Q3. What is the difference between git fetch and git pull?

Answer: `git fetch` downloads remote updates without integrating them into the current branch. `git pull` fetches updates and integrates them, usually through fast-forward or merge unless configured otherwise.

3. GitHub Integration with Jenkins

1. Definition — What is it?

GitHub integration with Jenkins means connecting Jenkins to a GitHub repository so Jenkins can retrieve source code and automate builds, tests, and deployments. GitHub stores source code; Jenkins executes automation.

2. Purpose — Why do we use it?

- Automatically retrieve application code.
- Build applications when code changes.
- Run tests before deployment.
- Support continuous integration and continuous delivery.
- Make build failures and results visible to the team.

3. Working — How does it work?

Configure the repository URL and branch in the Jenkins job.

Configure authentication when required, using Jenkins Credentials.

Jenkins checks out the selected source code and runs configured steps.

Builds can start manually, on a schedule, through SCM polling, or from webhook events.

4. Practical Example — How do we use it in a project?

Example repository: `https://github.com/Manjunathamanjuu/e2e-k8s-project1`.

In a Freestyle Job, select Git as Source Code Management, enter the repository URL, and use `*/master` as the branch specifier if master is the intended branch.

```
cd EKS-TF
terraform --version
terraform fmt -check -recursive
terraform init -backend=false
terraform validate
```

This assumes the checkout contains the EKS-TF directory at the repository root and required providers can be downloaded. The backend is skipped in this validation example; it is not a complete remote-state deployment pipeline.

For production, configure backend access and credentials securely. Never store AWS access keys, GitHub tokens, or private keys directly in the repository or Jenkinsfile.

5. Scenario-Based Interview Questions and Answers

Q1. Jenkins displays 'Failed to connect to repository'. How will you troubleshoot it?

Answer: Verify the repository URL, confirm whether it is public or private, check Jenkins credentials, test network and DNS connectivity from the Jenkins machine, verify Git installation and service-account access, and inspect Console Output.

Q2. Your GitHub repository is private. How will Jenkins access it?

Answer: Store a suitable credential in Jenkins Credentials. For HTTPS, use a fine-grained PAT with minimum required repository permissions; alternatively, use an SSH deploy key. Select the credential in the job and avoid exposing secrets in logs.

Q3. Jenkins checks out the wrong repository. What could cause this?

Answer: An incorrect repository URL, outdated job configuration, or wrong branch specification. Verify the job configuration and inspect the checkout URL and commit hash in the build log.

4. Branches in Git and Jenkins

1. Definition — What is it?

A Git branch is an independent line of development that allows developers to work on features, bug fixes, or releases without directly modifying another branch. Jenkins can build one branch or multiple branches depending on the job type.

2. Purpose — Why do we use it?

- Separate development from stable or production-ready code.
- Develop new features safely.
- Test changes before merging.
- Support parallel development.
- Trigger different CI/CD workflows for different branches.

3. Working — How does it work?

Create a feature or bug-fix branch from an appropriate starting point.

Commit and push changes to the remote branch.

Configure Jenkins to build a specific branch, or use a Multibranch Pipeline to discover eligible branches containing a Jenkinsfile.

Run checks and review results before merging or deploying.

4. Practical Example — How do we use it in a project?

For an EKS change, create and push a feature branch:

```
git branch
git switch -c feature/eks-update
git add .
git commit -m "Update EKS configuration"
git push -u origin feature/eks-update
```

In a Freestyle Job, `*/master` targets master, while `*/feature/eks-update` targets that feature branch.

To build multiple branches automatically, use a Multibranch Pipeline. It discovers eligible branches with a Jenkinsfile and creates a separate branch-specific job.

For infrastructure projects, run formatting, validation, and a carefully reviewed Terraform plan before applying changes. Restrict destructive operations and production deployment to authorized branches and approvals.

5. Scenario-Based Interview Questions and Answers

Q1. Jenkins builds master successfully, but your feature branch is not building. What will you check?

Answer: Check the branch specifier, confirm the feature branch exists on GitHub, verify triggers and checkout configuration, and determine whether the job is a single-branch Freestyle Job or a Multibranch Pipeline.

Q2. How do you configure Jenkins to build all eligible branches?

Answer: Configure a Multibranch Pipeline, set the repository source and credentials, and define branch discovery and filtering rules. Jenkins scans for branches containing a valid Jenkinsfile.

Q3. Should every feature branch automatically deploy to production?

Answer: No. Feature branches should normally run validation and tests. Production deployment should follow release policy, including approval, authorization, and environment-specific controls.

5. GitHub Webhooks

1. Definition — What is it?

A GitHub webhook is a mechanism through which GitHub sends an HTTP request containing event information to a configured endpoint when an event occurs, such as a push or pull request. Jenkins can receive these events and trigger jobs.

2. Purpose — Why do we use it?

- Trigger builds soon after a developer pushes code.
- Reduce unnecessary scheduled polling.
- Automate CI after pull requests or repository changes.
- Provide faster feedback to developers.
- Connect GitHub events to Jenkins pipelines.

3. Working — How does it work?

A developer pushes code to GitHub.

GitHub sends an HTTP POST request to the configured webhook endpoint.

Jenkins must be reachable and configured to handle the event.

The relevant job starts, checks out code, runs its build steps, and reports a result.

4. Practical Example — How do we use it in a project?

A typical webhook URL for the Jenkins GitHub plugin is:

```
https://your-public-jenkins-domain/github-webhook/
```

The actual URL depends on Jenkins configuration and plugin. In the Jenkins job, open Configure → Build Triggers and enable **GitHub hook trigger for GITScm polling** if using the GitHub plugin.

In GitHub, open repository Settings → Webhooks → Add webhook. Enter an endpoint GitHub can reach, use `application/json`, and select required events such as push.

A Jenkins server running at `http://localhost:8180` is not directly reachable by GitHub. Use a publicly reachable, securely configured endpoint or an appropriate secure network/tunnel solution. Do not expose Jenkins publicly without authentication, HTTPS, and suitable access restrictions.

To test, push a harmless change, check Jenkins build history, then inspect GitHub webhook delivery status and Jenkins logs if no build starts.

5. Scenario-Based Interview Questions and Answers

Q1. You push code to GitHub, but Jenkins does not start a build. How will you troubleshoot it?

Answer: Confirm the push and branch, verify webhook URL and selected events, check GitHub delivery history and HTTP response, confirm Jenkins is reachable, verify plugin and trigger configuration, and check branch filters.

Q2. The webhook returns a connection timeout. What is the likely problem?

Answer: GitHub may not be able to reach Jenkins because it is local, a firewall blocks traffic, DNS is incorrect, or the endpoint is unavailable. Verify reachability, endpoint, firewall rules, and reverse proxy or tunnel configuration.

Q3. Is a webhook the same as polling?

Answer: No. A webhook sends an event notification from GitHub to Jenkins. Polling means Jenkins periodically checks the repository for changes. Webhooks generally provide faster notification and reduce repeated repository checks.

6. Build Triggers in Jenkins

1. Definition — What is it?

A build trigger defines the condition or event that causes a Jenkins job to start. Jenkins supports manual execution, scheduled builds, SCM polling, webhooks, and upstream-job triggers.

2. Purpose — Why do we use it?

- Automate builds without manual intervention.
- Schedule routine tasks.
- Start CI when source code changes.
- Connect multiple jobs into a workflow.
- Reduce delays between code changes and testing.

3. Working — How does it work?

Jenkins monitors for a configured condition or receives an event.

When the trigger condition is met, Jenkins schedules the corresponding job.

The job checks out source code and runs the configured steps.

4. Practical Example — How do we use it in a project?

Common triggers include Build Now (manual), Build periodically (scheduled runs regardless of SCM changes), Poll SCM (check for changes on a schedule), GitHub webhook (react to repository events), and upstream-job completion.

To check a Terraform repository for changes approximately every five minutes, open Configure → Build Triggers → Poll SCM and enter:

```
H/5 * * * *
```

For a daily scheduled job, select Build periodically and enter:

```
H 2 * * *
```

The second expression schedules a run approximately once daily around 2 AM according to the Jenkins controller's configured time zone.

Build periodically schedules a run even when code has not changed. Poll SCM checks for source changes and normally triggers a build only when relevant changes are detected.

5. Scenario-Based Interview Questions and Answers

Q1. You want Jenkins to run every night, even when there are no code changes. Which trigger do you use?

Answer: Use Build periodically and configure the required cron expression, such as `H 2 * * *` for a daily run around 2 AM.

Q2. You want a job to start only when new code is pushed to GitHub. What would you configure?

Answer: Configure a GitHub webhook and enable the corresponding Jenkins build trigger. Verify that the webhook can reach Jenkins and the job uses the correct repository and branch.

Q3. Job A tests code and Job B deploys it. How can Job B start after Job A succeeds?

Answer: Configure Job B with an upstream-job trigger that runs when Job A completes successfully. Restrict deployment credentials and permissions appropriately.

7. Important Jenkins Cron Syntax

Jenkins scheduling commonly uses five fields:

```
MINUTE HOUR DAY-OF-MONTH MONTH DAY-OF-WEEK
```

Expression	Meaning
H/5 * * * *	Approximately every five minutes
H * * * *	Approximately once every hour
H 2 * * *	Daily around 2 AM
H 0 * * 1-5	Once per weekday around midnight
H 9 * * 1	Once per Monday around 9 AM

Note: H is a Jenkins-specific hash symbol that helps distribute job execution times. It does not mean zero or a fixed minute.

8. Real-Time Project Scenario: EKS Terraform CI/CD

Example workflow for an EKS Terraform project:

- **1. Developer creates a branch:** `feature/eks-update`.
- **2. Developer pushes code:** GitHub stores the commit.
- **3. Webhook triggers Jenkins:** A configured job receives the event.
- **4. Jenkins runs Terraform checks:** format, initialize, validate, and plan.
- **5. Review and approve:** Review the plan before infrastructure changes.

Sample validation sequence for a Jenkins agent with Terraform installed:

```
cd EKS-TF
terraform fmt -check -recursive
terraform init -backend=false
terraform validate
```

This assumes the checkout contains EKS-TF at the repository root and required providers can be downloaded. The backend is skipped in this example, so this is not a complete remote-state deployment pipeline. For a real infrastructure pipeline, initialize the configured backend, use the correct workspace and variable values, run terraform plan, and require appropriate approval before terraform apply. Do not run apply simply because validation succeeded.

9. Quick Interview Revision — Practice Quiz

1. Which Jenkins job is configured mainly through the web interface?

Freestyle Job — configured through Jenkins job settings and build steps.

2. Which Git command uploads committed changes to GitHub?

git push — sends local commits to the configured remote repository.

3. How can GitHub notify Jenkins about a push event?

GitHub webhook — sends an HTTP event notification to a reachable endpoint.

4. Which job type is suitable for discovering multiple branches with Jenkinsfiles?

Multibranch Pipeline — discovers eligible repository branches and creates branch-specific jobs.

5. Which trigger schedules a run even when code has not changed?

Build periodically — schedules runs according to its cron expression regardless of SCM changes.

6. A Jenkins job checks out the wrong branch. What should you check first?

Check the branch specifier and repository URL.

10. Most Important Interview Questions to Prepare

- What is a Freestyle Job, and when would you use a Pipeline instead?
- Explain how Jenkins connects to a GitHub repository.
- What is the difference between Git and GitHub?
- What is the difference between a branch and a repository?
- How do you configure Jenkins to build a specific branch?
- What is a GitHub webhook, and how does it trigger Jenkins?
- Why does a webhook fail when Jenkins is running on localhost?
- What is the difference between Poll SCM and Build periodically?
- What does H/5 * * * * mean in Jenkins?
- How would you troubleshoot a failed Git checkout?
- How do you securely store GitHub and AWS credentials in Jenkins?
- How would you prevent unreviewed Terraform changes from reaching production?

Interview tip: For scenario-based questions, explain the problem, the checks you would perform, the corrective action, and how you would verify the fix. This demonstrates practical troubleshooting rather than memorized definitions.